

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0515

Slot B

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 20%

QUESTIONS:5

Total Marks 20

Duration :60 Minutes

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I S.Jeswanth Kumar (192321056) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S Jeswanth Kumar

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Object Detection Pipeline

A system must:

- A. Capture video frames
- B. Detect objects using a trained model
- C. Draw bounding boxes and labels
- D. Display results in real time

Constraints:

- Must process frames efficiently
- Handle multiple objects

Write detailed pseudocode with modular steps for detection, visualization, and efficient looping.

2. Image Difference Detection System

A system must:

- A. Read two images
- B. Compare them
- C. Highlight differences

Constraints:

- Handle noise
- Ensure accurate difference detection

Write pseudocode including preprocessing, comparison logic, and visualization.

3. Video Frame Extraction System

A system must:

- A. Read video input
- B. Extract frames at fixed intervals
- C. Save selected frames

Constraints:

- Efficient storage
- Avoid redundant frames

Write structured pseudocode for frame selection and saving logic.

4. Background Subtraction Pipeline

A surveillance system must:

- A. Read video stream
- B. Separate foreground from background
- C. Highlight moving objects

Constraints:

- Handle dynamic background
- Reduce noise

Write pseudocode including background modeling, subtraction, filtering, and display.

5. Edge Detection Pipeline for Real-Time Video

A system must:

- A. Capture video frames

B. Apply edge detection

C. Display results

Constraints:

- Real-time processing
- Maintain clarity of edges

Write pseudocode with preprocessing, edge detection, and visualization modules.

ANSWERS

1. Real-Time Object Detection Pipeline

Pseudocode

```
ALGORITHM RealTimeObjectDetection
BEGIN
    INITIALIZE trained object detection model
    INITIALIZE video camera
    SET confidence_threshold = 0.5
    WHILE camera is active DO
        frame ← CAPTURE_FRAME()
        IF frame is not available THEN
            BREAK
        END IF
        resized_frame ← RESIZE(frame)
        detections ← DETECT_OBJECTS(resized_frame, model)
        FOR each detection IN detections DO
            IF detection.confidence ≥ confidence_threshold THEN
                x, y, width, height ← detection.bounding_box
                label ← detection.class_name
                DRAW_RECTANGLE(resized_frame, x, y, width, height)
                DRAW_LABEL(resized_frame, label, detection.confidence)
            END IF
        END FOR
        DISPLAY(resized_frame)
        IF user presses ESC THEN
            BREAK
        END IF
    END WHILE
    RELEASE camera
    CLOSE display window
END
```

Modular Operations

```
FUNCTION DETECT_OBJECTS(frame, model)
    preprocess frame
    detections ← model.predict(frame)
    RETURN detections
END FUNCTION

FUNCTION DRAW_LABEL(frame, label, confidence)
    text ← label + confidence
    DRAW text on frame
END FUNCTION
```

The frame is resized before detection to reduce computation. A confidence threshold removes unreliable detections. The loop processes each frame only once and supports multiple detected objects through iteration over all detections.

2. Image Difference Detection System

Pseudocode

```
ALGORITHM ImageDifferenceDetection
BEGIN
  image1 ← READ_IMAGE("image1")
  image2 ← READ_IMAGE("image2")
  IF image1 or image2 is not available THEN
    DISPLAY "Image loading error"
    STOP
  END IF
  image1 ← RESIZE_TO_SAME_SIZE(image1)
  image2 ← RESIZE_TO_SAME_SIZE(image2)
  gray1 ← CONVERT_TO_GRAYSCALE(image1)
  gray2 ← CONVERT_TO_GRAYSCALE(image2)
  filtered1 ← APPLY_GAUSSIAN_BLUR(gray1)
  filtered2 ← APPLY_GAUSSIAN_BLUR(gray2)
  difference ← ABSOLUTE_DIFFERENCE(filtered1, filtered2)
  binary_difference ← APPLY_THRESHOLD(difference)
  binary_difference ← MORPHOLOGICAL_OPENING(binary_difference)
  binary_difference ← MORPHOLOGICAL_CLOSING(binary_difference)
  contours ← FIND_CONTOURS(binary_difference)
  result ← COPY(image2)
  FOR each contour IN contours DO
    IF CONTOUR_AREA(contour) > minimum_area THEN
      x, y, width, height ← BOUNDING_RECTANGLE(contour)
      DRAW_RECTANGLE(result, x, y, width, height)
    END IF
  END FOR
  DISPLAY result
END
```

Gaussian filtering reduces noise before comparison. Absolute difference identifies changed pixels, while thresholding converts the difference into a binary image. Morphological operations remove small noise and connect relevant regions. Contour-area filtering ensures that insignificant changes are not highlighted.

3. Video Frame Extraction System

Pseudocode

```
ALGORITHM VideoFrameExtraction
BEGIN
  video ← OPEN_VIDEO("input_video")
  frame_interval ← SET_INTERVAL()
  frame_number ← 0
  saved_number ← 0
```

```

WHILE video has frames DO
    frame ← READ_NEXT_FRAME(video)
    IF frame is not available THEN
        BREAK
    END IF
    IF frame_number MOD frame_interval = 0 THEN
        frame ← REMOVE_REDUNDANT_INFORMATION(frame)
        filename ← CREATE_FILENAME(saved_number)
        SAVE_IMAGE(frame, filename)
        saved_number ← saved_number + 1
    END IF
    frame_number ← frame_number + 1
END WHILE
RELEASE video
END

```

The system extracts only frames at a fixed interval instead of saving every frame. The modulo operation determines whether the current frame should be selected. This reduces storage requirements and avoids highly similar consecutive frames.

4. Background Subtraction Pipeline

Pseudocode

```

ALGORITHM BackgroundSubtraction
BEGIN
    video ← OPEN_VIDEO("surveillance_video")
    background_model ← INITIALIZE_BACKGROUND_MODEL()
    WHILE video is active DO
        frame ← READ_NEXT_FRAME(video)
        IF frame is not available THEN
            BREAK
        END IF
        resized_frame ← RESIZE(frame)
        foreground_mask ← SUBTRACT_BACKGROUND(
            background_model,
            resized_frame
        )
        foreground_mask ← APPLY_THRESHOLD(foreground_mask)
        foreground_mask ← APPLY_MORPHOLOGICAL_OPENING(
            foreground_mask
        )
        foreground_mask ← APPLY_MORPHOLOGICAL_CLOSING(
            foreground_mask
        )
        contours ← FIND_CONTOURS(foreground_mask)
        result ← COPY(resized_frame)
        FOR each contour IN contours DO
            IF CONTOUR_AREA(contour) > minimum_area THEN
                x, y, width, height ← BOUNDING_RECTANGLE(contour)
                DRAW_RECTANGLE(
                    result,
                    x, y, width, height
                )
            )
        )
    )
END

```

```

        END IF
    END FOR
    DISPLAY(result)
    UPDATE_BACKGROUND_MODEL(background_model, resized_frame)
    IF user presses ESC THEN
        BREAK
    END IF
END WHILE
RELEASE video
CLOSE display window
END

```

An adaptive background model is used to handle gradual changes in the environment. Thresholding separates foreground pixels from the background. Morphological opening removes noise, while closing fills small gaps in moving objects. Contour-area filtering prevents small environmental changes from being treated as moving objects.

5. Edge Detection Pipeline for Real-Time Video

Pseudocode

```

ALGORITHM RealTimeEdgeDetection
BEGIN
    video ← INITIALIZE_CAMERA()
    WHILE camera is active DO
        frame ← CAPTURE_FRAME()
        IF frame is not available THEN
            BREAK
        END IF
        resized_frame ← RESIZE(frame)
        gray_frame ← CONVERT_TO_GRAYSCALE(resized_frame)
        smoothed_frame ← APPLY_GAUSSIAN_BLUR(
            gray_frame
        )
        edges ← APPLY_CANNY_EDGE_DETECTION(
            smoothed_frame,
            lower_threshold,
            upper_threshold
        )
        DISPLAY(edges)
        IF user presses ESC THEN
            BREAK
        END IF
    END WHILE
    RELEASE camera
    CLOSE display window
END

```

Gaussian smoothing is applied before edge detection to reduce noise and prevent false edges. The Canny edge detector identifies strong intensity changes while preserving important boundaries. Resizing the frame reduces processing time and helps maintain real-time performance. Appropriate Canny thresholds are selected to maintain clear and continuous edges without excessive noise.